



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

SECP3133 – HIGH PERFORMANCE DATA PROCESSING

*Project 1: Optimizing High Performance Data Processing
for Large Scale Web Crawlers*

Programme	: Bachelor of Computer Science (Data Engineering)
Subject Code	: SECP3133
Subject Name	: High Performance Data Processing
Session-Sem	: 2025/2026-02

Prepared by : 1) Muhammad Afiq Danish bin Mohd Hazni (A23CS0118)
2) Muhammad Syahmi Faris bin Rusli (A23CS0138)
3) Ng Yu Hin (A23CS0148)

Section : 01

Lecturer : Prof. Madya. Ts. Dr. Mohd Shahizan bin Othman

Table of Contents

1.0 Introduction	2
2.0 System Design and Architecture	2
3.0 Data Collection	2
4.0 Data Processing	2
5.0 Optimization Techniques	3
5.1 Baseline Pandas	3
5.2 Multiprocessing	4
5.3 Polars	6
6.0 Performance Evaluation	8
6.1 Performance Measurement Function	8
6.2 Results	8
6.3 Results Analysis	10
7.0 Challenges and Limitations	11
8.0 Conclusion	11

1.0 Introduction

2.0 System Design and Architecture

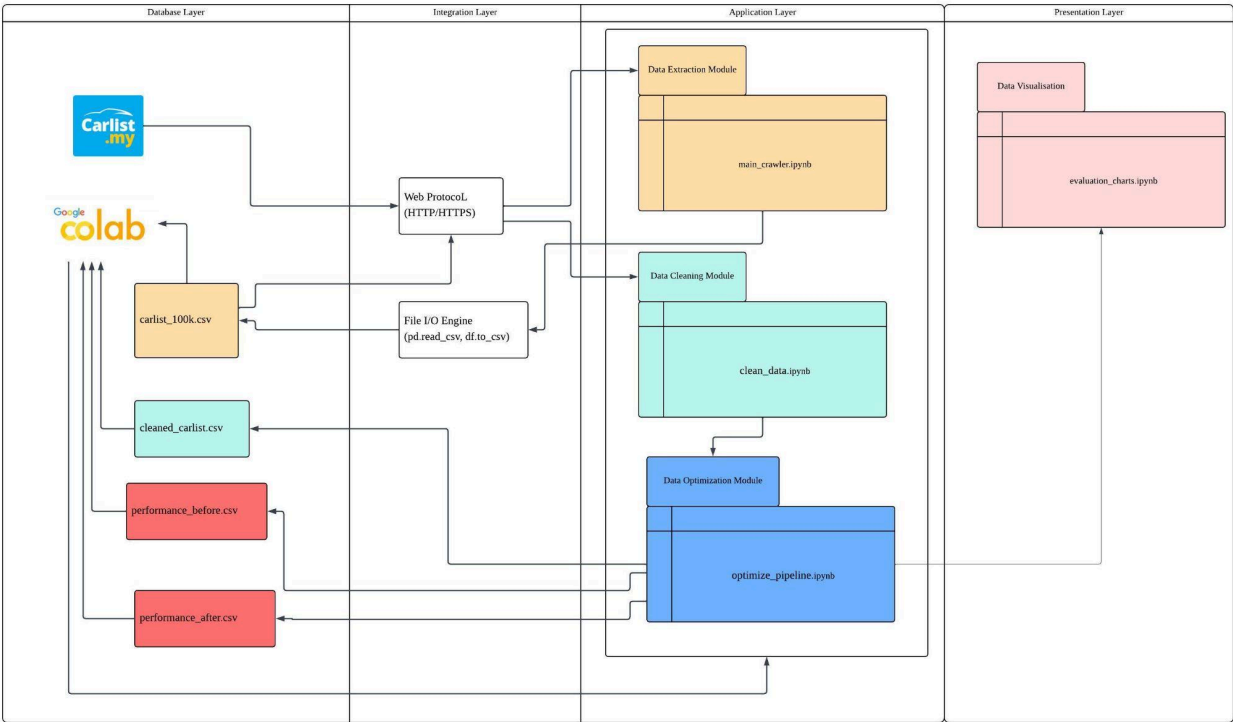


Figure 1: System Architecture

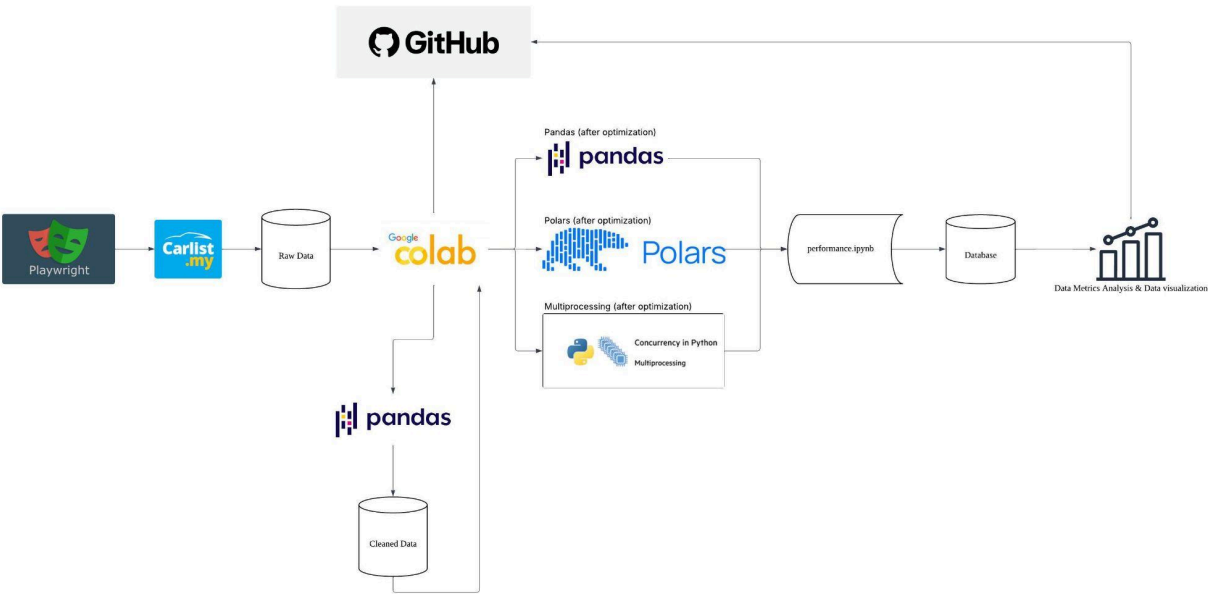


Figure 2: Architecture of Tools and Frameworks used

3.0 Data Collection

The dataset was collected from [Carlist.my](https://www.carlist.my/) (<https://www.carlist.my/>), one of Malaysia's largest online car marketplaces, by crawling its national listings index. A total of 150,015 raw records were gathered, each with 11 attributes.

Crawling Method. The crawler was built in Python using Playwright, a headless browser framework. A browser-based approach was necessary because [Carlist.my](https://www.carlist.my/) renders its listings via JavaScript, so a plain HTTP request would return no usable data. Key design features:

- **Asynchronous Concurrency:** uses asyncio with a semaphore limiting the crawler to 4 concurrent browser context, fetching multiple pages in parallel.
- **Pagination Traversal:** iterates through listing pages (25 per page) using `page_number` query parameters until the target count is reached.
- **Direct DOM Extraction:** all 11 fields are extracted in a single in-page `page.evaluate()` call, reading each article listing card for title, year, price, mileage and other attributes. Condition (USED/RECON/NEW) is inferred from the URL.
- **Resilience:** each request retries up to 3 times with exponential backoff, handling HTTP 403/429 rate limit responses, fonts, images, and analytics requests are blocked to speed up loading.
- **Resumability:** progress and seen IDs are persisted, so the crawler can resume after interruption and skip duplicates.

Ethical Considerations. Only publicly accessible listing data was collected, no login or personal data. Server load was kept modest through limited concurrency, a 1.5 second delay per page, and backoff on rate limiting. The data is used strictly for academic purposes and is not redistributed.

4.0 Data Processing

The raw dataset contained inconsistencies typical web-scraped data which is numbers stored as strings, noisy text fields, compound fields, missing values, and outliers. A cleaning pipeline was built in a Jupyter Notebook which is `clean_data.ipynb` using Pandas to produce the analysis ready `carlist_cleaned.csv`.

Raw Data Structure: The raw file have 150,015 rows x 11 columns. Issues included price and `monthly_payment` as currency strings, mileage as a range string, `seller_type` as noisy text, location combining state and city, and unrealistic year values. Missing values appeared in `monthly_payment` (187), `seller_type` (55), and `id` (1).

Cleaning Steps:

1. **Duplicate removal:** dropped exact duplicate rows
2. **Price parsing:** RM 63,999 changed into 63999.0, a filter (RM1,000 - RM5,000,000) removed 112 outliers
3. **Monthly payment parsing:** converted to numeric, around 187 missing values imputed with the median
4. **Mileage parsing:** range string converted to a numeric midpoint (75 - 80 KM to 77,500)
5. **Seller type extraction:** reduced noisy text to clean labels (Dealer, Sales Agent, Private, Broker and Other)
6. **Location splitting:** split into separate state and city columns
7. **Year filtering:** kept 1980 - 2026, removing 21 outliers
8. **Categorical standardisation:** condition and transmission trimmed and Title-cased
9. **Schema finalisation:** dropped redundant raw columns, leaving a 12-column schema

Final Data Structure: After removing 133 records in total, the final dataset contains 159,882 rows x 12 columns: id, title, year, price_myr, monthly_payment_myr, mileage_km, transmission, seller_type, state, city, condition, and url. This cleaned dataset services as the common input for all three optimisation techniques in Section 5.

5.0 Optimization Techniques

This section describes the two optimization techniques applied to the data processing pipeline alongside the baseline approach used for comparison. All three methods execute a similar analytical pipeline on the same dataset which is `carlist_cleaned.csv` containing 149,882 records and 12 features so that performance differences can be attributed solely to the technique being used.

The process benchmarked in each method consists of five stages:

1. Load the CSV file.
2. Filter rows to select only used cars with mileage below 100,000 km.
3. Feature engineering process which computes three derived features (*price_per_km*, *car_age*, *price_per_year*).
4. Aggregations by state and transmission type.
5. Finding top 10 best value cars

5.1 Baseline Pandas

The baseline uses the standard Pandas library with no parallelism. All operations execute sequentially on a single CPU core which is the most common approach in typical Python workflows.

The pipeline loads the dataset using `pd.read_csv()`, applies boolean conditioning for row filtering, and creates new columns through direct arithmetic. Groupby aggregations are then performed using `.groupby().agg()`. The top 10 best value cars are identified using `nsmallest(10, 'price_per_km')`, which selects the 10 rows with the lowest price-to-mileage ratio. It represents cars that offer the most distance covered per ringgit spent. Because Pandas processes all of these steps sequentially in a single thread, execution is bounded by the speed of one core and the overhead of Python's garbage-collected memory model.

```

def baseline_analysis(csv_path):
    # — Step 1: Load data —————
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # — Step 2: Filter —————
    df_f = df[
        (df['mileage_km'] < 100_000) &
        (df['condition'] == 'Used') &
        (df['price_myr'] > 0)
    ].copy()

    # — Step 3: Feature Engineering —————
    df_f['price_per_km'] = df_f['price_myr'] / (df_f['mileage_km'] + 1)
    df_f['car_age'] = CURRENT_YEAR - df_f['year']
    df_f['price_per_year'] = df_f['price_myr'] / (df_f['car_age'] + 1)

    # — Step 4: Aggregate by state —————
    state_stats = df_f.groupby('state').agg(
        avg_price = ('price_myr', 'mean'),
        car_count = ('id', 'count'),
        avg_mileage = ('mileage_km', 'mean'),
        median_price = ('price_myr', 'median')
    ).reset_index().sort_values('avg_price', ascending=False)

    # — Step 5: Aggregate by transmission —————
    trans_stats = df_f.groupby('transmission').agg(
        avg_price = ('price_myr', 'mean'),
        car_count = ('id', 'count'),
        avg_mileage = ('mileage_km', 'mean')
    ).reset_index()

    # — Step 6: Top 10 best value cars —————
    top_value = df_f.nsmallest(10, 'price_per_km')[
        ['title', 'state', 'price_myr', 'mileage_km', 'price_per_km']
    ]

    return total_rows, len(df_f), state_stats, trans_stats, top_value

```

Figure 1. Code Snippet of Pipeline Execution Using Pandas

This baseline establishes the reference performance figures against which both optimization techniques are measured.

5.2 Multiprocessing

The first optimization technique used is Python's multiprocessing module. The idea behind this technique is to split the dataset into smaller pieces and process each piece at the same time using different CPU cores. Since the machine used has 16 CPU cores, the dataset was split into 16 chunks and each chunk was sent to a different worker process.

The reason multiprocessing was chosen instead of multithreading is because of the Global Interpreter Lock, or GIL. The GIL is a limitation in Python that stops multiple threads from running Python code at the same time. This means that even when multiple threads are used, it cannot actually run in parallel for CPU-heavy tasks like calculations. Multiprocessing gets around this problem by using separate processes instead of threads, so each process has its own Python interpreter and there is no GIL limitation.

One issue that was encountered is that the worker functions cannot be defined inside the Jupyter Notebook. This is because on Windows, Python creates new processes by re-importing the main script, and when the main script is a Jupyter Notebook, this causes the program to hang and never finish.

```
def process_chunk(args):
    """
    Runs in each worker process.
    Applies filter + feature engineering on one DataFrame chunk.
    """
    chunk_df, _ = args

    filtered = chunk_df[
        (chunk_df['mileage_km'] < 100_000) &
        (chunk_df['condition'] == 'Used') &
        (chunk_df['price_myr'] > 0)
    ].copy()

    filtered['price_per_km'] = filtered['price_myr'] / (filtered['mileage_km'] + 1)
    filtered['car_age'] = CURRENT_YEAR - filtered['year']
    filtered['price_per_year'] = filtered['price_myr'] / (filtered['car_age'] + 1)

    return filtered
```

Figure 2. Code Snippet of *mp_helpers.py*

To fix this, the worker function was moved into a separate file called *mp_helpers.py*. The *joblib* library with the *loky* backend was also used to manage the worker processes, because it handles this situation better than using *multiprocessing.Pool* directly.

```

def multiprocessing_analysis(csv_path, n_workers):
    # — Step 1: Load data (serial – file I/O) —————
    df = pd.read_csv(csv_path)
    total_rows = len(df)

    # — Step 2: Split DataFrame into N chunks (pandas iloc) –
    chunks = chunk_dataframe(df, n_workers)
    args = [(chunk, i) for i, chunk in enumerate(chunks)]

    # — Step 3: Process chunks in parallel —————
    # joblib loky backend avoids the Windows+Jupyter spawn hang
    results = Parallel(n_jobs=n_workers, backend='loky')(
        delayed(process_chunk)(arg) for arg in args
    )

    # — Step 4: Merge results from all workers —————
    df_f = pd.concat(results, ignore_index=True)

```

Figure 3. Code Snippet of Pipeline Execution Using Multiprocessing

After all the worker processes finish, the results are combined back together using *pd.concat()* in the main process. Then, the aggregation steps are run on the combined data.

5.3 Polars

The second optimization technique used is Polars which is a DataFrame library written in a programming language called Rust. Polars works differently from Pandas because it does not run each operation immediately. Instead, it first builds a plan of all the steps that need to be done, then optimizes that plan and only then actually runs everything at once. This is called lazy evaluation.

Polars is used by calling *pl.scan_csv()* instead of reading the file directly. This does not load the file yet. After that, filter and feature engineering steps are added to the plan. When *.collect()* is finally called, Polars looks at the full plan, figures out the most efficient way to run it and then executes everything using multiple CPU cores automatically. There is no need to manually split the data or create worker processes like in the multiprocessing approach.

Polars also stores data in a format called Apache Arrow which organizes data by columns instead of by rows. This makes operations like filtering and calculating averages faster because the CPU only needs to read the relevant columns instead of going through every row.

```

def polars_analysis(csv_path):
    # — Step 1: Lazy scan – file is NOT loaded yet —————
    # Polars builds a query plan and optimizes before executing
    lf = pl.scan_csv(csv_path)

    # — Step 2: Filter (lazy) —————
    lf_f = lf.filter(
        (pl.col('mileage_km') < 100_000) &
        (pl.col('condition') == 'Used') &
        (pl.col('price_myrr') > 0)
    )

    # — Step 3: Feature engineering (lazy) —————
    lf_f = lf_f.with_columns([
        (pl.col('price_myrr') / (pl.col('mileage_km') + 1)).alias('price_per_km'),
        (CURRENT_YEAR - pl.col('year')).alias('car_age'),
        (pl.col('price_myrr') / (CURRENT_YEAR - pl.col('year') + 1)).alias('price_per_year')
    ])

    # — Step 4: Collect – executes the full optimized plan –
    df_f = lf_f.collect()
    total_rows = pl.scan_csv(csv_path).select(pl.len()).collect().item()
    filtered_rows = len(df_f)

```

Figure 4. Code Snippet of Pipeline Execution Using Polars

Compared to multiprocessing, Polars is easier to use because there is no need to manually manage processes or worry about how data is passed between workers. The parallelism happens inside Polars itself without any extra setup required.

6.0 Performance Evaluation

6.1 Performance Measurement Function

To measure the performance of each method, a tool called *MetricsCollector* was built. It is a context manager which means it can be wrapped around any block of code and it will automatically record the performance while that code is running. Four metrics were measured for each method:

- Execution time is how long the code takes to finish, measured using *time.perf_counter()* which is a very accurate timer.
- Peak memory is the highest amount of memory used during the run. This was measured using two tools together, *tracemalloc* which tracks memory used by Python and *psutil* which also tracks memory used by non-Python parts like Rust or C libraries.
- Average CPU utilization is how much of the CPU was being used on average. This was checked every 0.1 seconds using *psutil.cpu_percent()* running in a background and then the average was calculated.
- Throughput is how many rows the method can process per second, calculated by dividing the total number of rows by the time taken.

Each method was also run five times and the average of the results was taken. This was done because the first run is sometimes slower due to the computer needing to load the file into memory for the first time. By averaging five runs, a more accurate and fair result is obtained.

6.2 Results

The table below shows the average performance results for all three methods across five runs.

Metric	Pandas	Multiprocessing	Polars
Execution Time (s)	0.631	0.906	0.059
Peak Memory (MB)	66.8	99.6	33.8
Average CPU Utilization (%)	25.4	23.3	30.7
Throughput (rows/s)	237753	165629	2547055

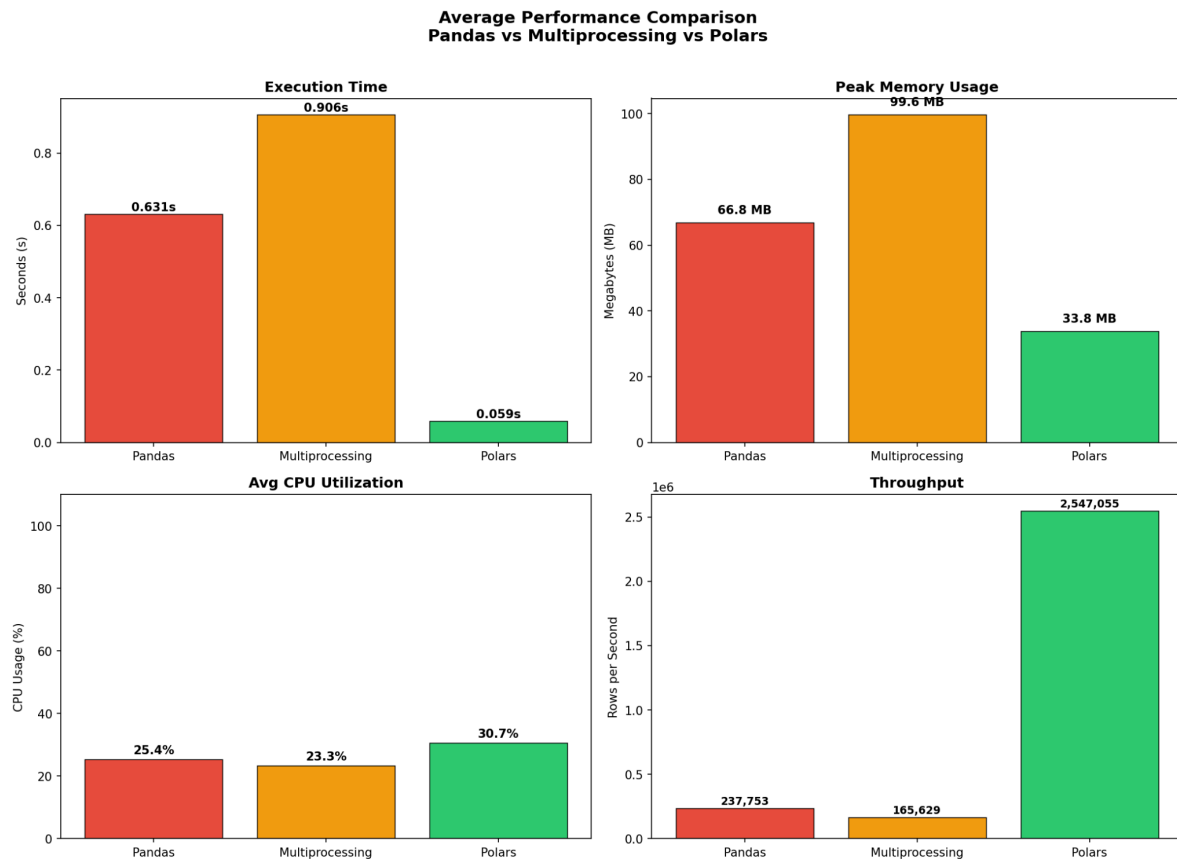


Figure 5. Performance Comparison Charts

Figure 5 shows the performance comparison charts for all three methods across the four metrics. Looking at the execution time chart, Polars is the fastest method at 0.059 seconds while Multiprocessing is the slowest at 0.906 seconds, which is even higher than the Pandas baseline at 0.631 seconds. This shows that having more CPU cores does not always mean the program will run faster.

For peak memory usage, Multiprocessing used the most memory at 99.6 MB which is almost three times more than Polars at 33.8 MB. Pandas falls in the middle at 66.8 MB. The high memory usage of Multiprocessing is because copies of the data need to be made for each of the 16 worker processes.

The CPU utilization chart shows that all three methods have similar CPU usage levels. Polars had the highest at 30.7%, followed by Pandas at 25.4%, and Multiprocessing at 23.3%. Even though Multiprocessing uses 16 worker processes, its average CPU usage is actually the lowest, which suggests that the workers were spending more time waiting for data than actually doing computation.

The throughput chart shows the biggest difference between the methods. Polars processed approximately 2,547,055 rows per second which is about 10.7 times more than Pandas at 237,753 rows per second, and about 15.4 times more than Multiprocessing at 165,629 rows per second. This clearly shows that Polars is the most efficient method for processing this dataset.

6.3 Results Analysis

Execution Time: Polars finished the pipeline in 0.059 seconds on average which is about 10.7 times faster than Pandas at 0.631 seconds. Multiprocessing took 0.906 seconds which is actually 43.6% slower than Pandas even though it was using all 16 CPU cores. This happened because the dataset only has about 149,882 rows as it is relatively small. Before any actual processing can happen, Multiprocessing needs to split the data into 16 chunks, send each chunk to a separate worker process and then combine all the results back together at the end. For a dataset of this size, the time spent on these extra steps is more than the time saved by running things in parallel.

Memory Usage: Polars used the least memory at 33.8 MB which is about 49.4% less than Pandas at 66.8 MB and about 66% less than Multiprocessing at 99.6 MB. The reason Polars uses less memory is because of its lazy evaluation approach since it filters out unwanted rows before fully loading the data, there is less data that needs to be stored in memory at any one time. Multiprocessing uses the most memory because the dataset has to be copied and sent to each of the 16 worker processes separately that results in many copies of the data being stored in memory at the same time.

CPU Utilization: The average CPU usage was fairly similar across all three methods, ranging from 23.3% to 30.7%. Polars had the highest at 30.7%, which shows that it was making use of multiple CPU cores through its internal thread pool. Multiprocessing had the lowest at 23.3%, which is surprising for a method that uses 16 worker processes. This suggests that most of the time was spent on coordinating between processes rather than on doing actual computation.

Throughput: Polars processed about 2.55 million rows per second compared to 237,753 rows per second for Pandas and 165,629 rows per second for Multiprocessing. This means Polars is about 10.7 times faster than Pandas and about 15.4 times faster than Multiprocessing in terms of data processed per second. This is the clearest indicator of how much more efficient Polars is compared to the other two methods.

7.0 Challenges and Limitations

8.0 Conclusion